



密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: May 13, 2026

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	3
3.1	CBC 模式解密关系	3
3.2	PKCS#7 填充规则	3
3.3	Padding Oracle 的泄露点	4
3.4	利用填充反馈恢复中间状态	4
3.5	攻击本质	5
4	目标密文与分组	5
4.1	目标密文	5
4.2	密文分组	5
5	攻击算法设计	6
5.1	整体攻击思路	6
5.2	单个分组的攻击过程	6
5.3	伪造分组的构造方法	7
5.4	实际提交密文格式	8
5.5	算法伪代码	8
5.6	算法复杂度分析	9
5.7	算法特点	9
6	代码实现说明	9
7	运行结果与破解明文	9
7.1	Oracle 可用性测试	10
7.2	明文分组恢复结果	10
7.3	完整明文恢复结果	11
7.4	运行过程截图说明	11
8	实验分析与总结	11

1 实验目的

本次实验的目的是理解并实现针对 CBC 工作模式下 PKCS#7 填充机制的 Padding Oracle 攻击。通过实验，掌握在未知 AES 密钥的情况下，如何利用解密服务器对填充是否合法的反馈信息，逐字节恢复密文对应的明文。

具体而言，本实验需要完成以下目标：

1. 理解 AES-128-CBC 模式的加解密结构，明确明文分组、密文分组和初始向量 IV 之间的关系；
2. 掌握 PKCS#7 填充规则，理解填充合法性判断如何泄露明文相关信息；
3. 编写 Padding Oracle 攻击程序，利用服务器返回的填充状态恢复 CBC 解密过程中的中间状态；
4. 对本小组抽到的 5 号密文进行破解，得到完整明文；
5. 通过实验分析 CBC 模式在缺少完整性保护时可能产生的安全风险，加深对“加密不等于认证”的理解。

2 实验环境

本实验在本地编写攻击程序，并通过访问实验指导中给出的解密服务器完成 Padding Oracle 攻击。实验过程中，攻击者不需要知道 AES 密钥，只需要根据服务器返回的填充状态判断密文解密后的 PKCS#7 填充是否合法。

Table 1: 实验环境

项目	配置或说明
操作系统	Windows 11
编程语言	Python 3
开发工具	Visual Studio Code
加密算法	AES-128-CBC
填充标准	PKCS#7
目标密文编号	5 号密文
密钥组	Key_2
解密接口	http://10.102.33.67:8208/dec_2?data=

其中，实验指导给出的解密服务器会对输入的密文进行 AES-128-CBC 解密，并检查解密后明文是否满足 PKCS#7 填充规则。若填充正确，服务器返回 HTTP 200；若填充错误，服务器返回 HTTP 500 server error。因此，本实验程序通过构造不同的密文并观察服务器返回状态，逐步恢复目标密文对应的明文。

由于本小组抽到的是 5 号密文，而实验材料中 5 号密文属于 Key_2 加密得到的密文，因此程序中访问的解密接口为 dec_2。实验运行时需要保证本机能够访问学校实验服务器，通常需要处于校园网环境或连接学校 VPN。

3 实验原理

本实验的核心原理是 **Padding Oracle** 攻击。该攻击针对采用 **CBC** 工作模式并使用 **PKCS#7** 填充的分组密码系统。当解密服务器会向攻击者反馈“填充是否合法”时，即使攻击者不知道 **AES** 密钥，也可以通过构造密文并反复查询服务器，逐字节恢复目标密文对应的明文。

3.1 CBC 模式解密关系

在 **CBC** 模式中，设初始向量为 IV ，密文分组为 $C_1, C_2, \dots, C_n$ ，对应明文分组为 $P_1, P_2, \dots, P_n$ 。令 $C_0 = IV$ ，则第 i 个密文分组的 **CBC** 解密关系为：

$$P_i = D_K(C_i) \oplus C_{i-1} \quad (1)$$

其中， $D_K(C_i)$ 表示在密钥 K 下对密文分组 C_i 进行 **AES** 解密得到的中间状态， $\oplus$ 表示按字节异或运算。

为了便于描述，记：

$$I_i = D_K(C_i) \quad (2)$$

则有：

$$P_i = I_i \oplus C_{i-1} \quad (3)$$

因此，如果能够恢复出中间状态 I_i ，再与真实的前一密文分组 C_{i-1} 异或，就可以得到真实明文分组 P_i 。

3.2 PKCS#7 填充规则

AES 的分组长度为 16 字节，而明文长度不一定正好是 16 字节的整数倍，因此需要使用填充机制。**PKCS#7** 填充规则规定：如果需要填充 t 个字节，则每个填充字节的值都为 t 。

例如，当需要填充 1 个字节时，填充值为：

$$01 \quad (4)$$

当需要填充 2 个字节时，填充值为：

$$02 \ 02 \quad (5)$$

当需要填充 6 个字节时，填充值为：

$$06 \ 06 \ 06 \ 06 \ 06 \ 06 \quad (6)$$

因此，解密服务器在解密后会检查最后若干字节是否符合 **PKCS#7** 填充规则。如果填充合法，则返回正确信息；如果填充不合法，则返回错误信息。

3.3 Padding Oracle 的泄露点

在本实验中，攻击者可以访问解密服务器。服务器对输入密文进行 AES-128-CBC 解密后，会检查解密结果是否满足 PKCS#7 填充规则：

- 若填充正确，服务器返回 HTTP 200；
- 若填充错误，服务器返回 HTTP 500 server error。

从表面上看，服务器没有直接返回明文或密钥。但它返回的填充状态本身已经泄露了关于解密结果的信息。攻击者可以不断构造不同的密文输入，根据服务器返回的 HTTP 状态码判断某种猜测是否正确。这种能够判断填充是否合法的服务器就称为 Padding Oracle。

3.4 利用填充反馈恢复中间状态

假设攻击目标是某个密文分组 C_i 。攻击者构造一个伪造的前一分组 R ，并向服务器提交：

$$R \parallel C_i \quad (7)$$

服务器解密最后一个分组后得到：

$$D_K(C_i) \oplus R = I_i \oplus R \quad (8)$$

由于 R 是攻击者可以控制的，攻击者就可以通过修改 R 的字节，使得 $I_i \oplus R$ 的最后若干字节变成合法的 PKCS#7 填充。

例如，为了恢复最后一个字节，攻击者枚举 R 的最后一个字节 R_{15} ，直到服务器返回 HTTP 200。此时说明：

$$I_{i,15} \oplus R_{15} = 01 \quad (9)$$

于是可以得到：

$$I_{i,15} = R_{15} \oplus 01 \quad (10)$$

恢复出最后一个中间状态字节后，攻击者继续构造伪造分组，使最后两个字节变成：

$$02 \ 02 \quad (11)$$

然后枚举倒数第二个字节，恢复 $I_{i,14}$ 。依次类推，可以从后向前逐字节恢复整个中间状态：

$$I_i = D_K(C_i) \quad (12)$$

最后，根据 CBC 解密关系：

$$P_i = I_i \oplus C_{i-1} \quad (13)$$

即可得到目标密文分组 C_i 对应的真实明文分组。

3.5 攻击本质

Padding Oracle 攻击并不是直接破解 AES 算法本身，也不是恢复 AES 密钥，而是利用了解密系统实现中的信息泄露。具体来说，服务器把“填充正确”和“填充错误”以不同的状态返回给攻击者，使攻击者能够通过大量选择密文查询，逐步推断 CBC 解密的中状态，最终恢复明文。

因此，本实验说明：只提供机密性而缺少完整性保护的 CBC 加密实现是不安全的。在实际系统中，不能将可区分的 padding 错误直接暴露给外部用户，更安全的做法是使用认证加密模式，或者在解密前先进行消息认证。

4 目标密文与分组

根据实验要求，目标密文序号由小组成员学号最后一位之和对 10 取模得到。本小组抽到的目标密文序号为 5。实验材料中给出的 5 号密文属于 Key_2 加密所得密文，因此本实验在攻击过程中使用 dec_2 解密接口进行 Padding Oracle 查询。

4.1 目标密文

实验材料中给出的 5 号目标密文为：

```
46307250616464316
e674f7261636c339ae0735429869542efc40dc3f4c170649463f719c5ddf4ce8c6d1ef0e5d4
```

该密文采用 AES-128-CBC 模式加密，并使用 PKCS#7 填充。由于 AES 的分组长度为 16 字节，因此需要按照每 16 字节，即每 32 个十六进制字符，对密文进行分组。

4.2 密文分组

目标密文的整体结构为：

$$IV \parallel C_1 \parallel C_2 \parallel C_3 \quad (14)$$

其中， IV 为初始向量， C_1, C_2, C_3 为三个密文分组。按照 16 字节分组后得到：

Table 2: 5 号目标密文分组结果

分组	十六进制表示
IV	46307250616464316e674f7261636c33
C_1	9ae0735429869542efc40dc3f4c170
C_2	649463f719c5ddf4ce8c6d1ef0e5d41a
C_3	c5d137629e3fe1340cfaad7e21d65d14

程序运行时首先对目标密文进行分组，输出结果如下：

```
IV = 46307250616464316e674f7261636c33
C1 = 9ae0735429869542efc40dc3f4c170
C2 = 649463f719c5ddf4ce8c6d1ef0e5d41a
C3 = c5d137629e3fe1340cfaad7e21d65d14
```

在 CBC 解密过程中, IV 可以视为第 0 个密文分组, 即 $C_0 = IV$ 。因此三个明文分组的恢复关系分别为:

$$P_1 = D_K(C_1) \oplus IV, \quad (15)$$

$$P_2 = D_K(C_2) \oplus C_1, \quad (16)$$

$$P_3 = D_K(C_3) \oplus C_2. \quad (17)$$

后续攻击过程的目标就是分别恢复 $D_K(C_1)$ 、 $D_K(C_2)$ 和 $D_K(C_3)$ 的中间状态, 再与对应的前一分组异或, 最终得到完整明文。

5 攻击算法设计

本实验的攻击算法围绕 CBC 解密关系和 PKCS#7 填充反馈展开。攻击者并不尝试恢复 AES 密钥, 而是通过构造选择密文并反复访问解密服务器, 逐字节恢复每个密文分组对应的中间状态, 进而恢复明文。

5.1 整体攻击思路

设目标密文为:

$$IV \parallel C_1 \parallel C_2 \parallel C_3 \quad (18)$$

对于任意一个目标密文分组 C_i , CBC 解密关系为:

$$P_i = D_K(C_i) \oplus C_{i-1} \quad (19)$$

令:

$$I_i = D_K(C_i) \quad (20)$$

则有:

$$P_i = I_i \oplus C_{i-1} \quad (21)$$

因此, 攻击的关键是恢复中间状态 I_i 。由于攻击者可以控制伪造的前一分组 R , 服务器解密最后一个分组时得到:

$$D_K(C_i) \oplus R = I_i \oplus R \quad (22)$$

如果 $I_i \oplus R$ 的末尾字节满足 PKCS#7 填充规则, 服务器返回 HTTP 200; 否则返回 HTTP 500。攻击者根据这个反馈逐字节恢复 I_i 。

5.2 单个分组的攻击过程

攻击一个分组时, 从最后一个字节开始向前恢复。设 AES 分组长度为 16 字节, 字节下标为 $0, 1, \dots, 15$ 。攻击过程如下:

1. 构造一个 16 字节的伪造前置分组 R ;

2. 从最后一个字节开始，令期望填充值为 01，枚举 R_{15} ；
3. 将构造后的密文提交给解密服务器；
4. 若服务器返回 HTTP 200，则说明最后一个字节满足：

$$I_{i,15} \oplus R_{15} = 01 \quad (23)$$

因此可恢复：

$$I_{i,15} = R_{15} \oplus 01 \quad (24)$$

5. 恢复最后一个字节后，继续恢复倒数第二个字节。此时需要将最后两个字节都构造成 02 02；
6. 依次类推，当恢复第 j 个字节时，令已经恢复出的后缀字节全部满足当前填充值 pad ；
7. 重复上述过程，直到恢复完整的 16 字节中间状态 I_i ；
8. 最后计算：

$$P_i = I_i \oplus C_{i-1} \quad (25)$$

得到真实明文分组。

5.3 伪造分组的构造方法

假设当前要构造的填充值为 pad ，正在恢复的位置为 $pos = 16 - pad$ 。对于已经恢复出来的后缀位置 $j > pos$ ，为了让解密结果的后缀字节均等于 pad ，需要设置：

$$R_j = I_{i,j} \oplus pad \quad (26)$$

对于当前待恢复的位置 pos ，程序枚举 R_{pos} 的所有 256 种可能取值。当服务器返回填充合法时，有：

$$I_{i,pos} \oplus R_{pos} = pad \quad (27)$$

因此：

$$I_{i,pos} = R_{pos} \oplus pad \quad (28)$$

恢复出 $I_{i,pos}$ 后，再与真实的前一密文分组字节异或，即可得到对应的明文字节：

$$P_{i,pos} = I_{i,pos} \oplus C_{i-1,pos} \quad (29)$$

5.4 实际提交密文格式

理论上，攻击某个目标分组 C_i 时，可以向服务器提交：

$$R \parallel C_i \quad (30)$$

其中 R 是攻击者构造的伪造前一分组。

但是在本实验环境中，服务器接口接收的是完整的 IV+密文 格式。因此在具体实现中，为了保证提交格式符合服务器要求，程序实际提交：

$$IV \parallel R \parallel C_i \quad (31)$$

此时， C_i 仍然是最后一个密文分组，服务器检查的最后一个明文分组为：

$$D_K(C_i) \oplus R \quad (32)$$

这仍然满足 Padding Oracle 攻击的条件。恢复得到的中间状态仍然是 $I_i = D_K(C_i)$ 。随后程序再使用真实的前一密文分组 C_{i-1} 计算真正明文：

$$P_i = I_i \oplus C_{i-1} \quad (33)$$

因此，对于三个密文分组，本实验分别进行如下计算：

$$P_1 = D_K(C_1) \oplus IV, \quad (34)$$

$$P_2 = D_K(C_2) \oplus C_1, \quad (35)$$

$$P_3 = D_K(C_3) \oplus C_2. \quad (36)$$

5.5 算法伪代码

本实验攻击算法可以概括为如下伪代码：

Listing 1: Padding Oracle 攻击算法伪代码

```

for each target block C_i:
    intermediate = [0] * 16
    plaintext = [0] * 16

    for pad from 1 to 16:
        pos = 16 - pad
        fake_prev = [0] * 16

        for each recovered suffix byte j > pos:
            fake_prev[j] = intermediate[j] xor pad

        for guess from 0 to 255:
            fake_prev[pos] = guess
            test_cipher = IV || fake_prev || C_i

            if oracle(test_cipher) == valid:
                intermediate[pos] = guess xor pad

```

```
        plaintext[pos] = intermediate[pos] xor
            real_prev_block[pos]
        break

    output plaintext
```

其中, `oracle(test_cipher)` 表示向解密服务器提交构造密文, 并根据服务器返回的 HTTP 状态码判断 PKCS#7 填充是否合法。若服务器返回 HTTP 200, 则认为填充合法; 若返回 HTTP 500, 则认为填充非法。

5.6 算法复杂度分析

对于每个字节, 攻击者最多枚举 256 种可能取值。每个 AES 分组有 16 个字节, 因此攻击一个分组最多需要:

$$16 \times 256 = 4096 \quad (37)$$

次 `oracle` 查询。

本实验目标密文除 IV 外共有 3 个密文分组, 因此最多需要:

$$3 \times 16 \times 256 = 12288 \quad (38)$$

次 `oracle` 查询。实际运行时, 平均每个字节通常不需要枚举满 256 次, 因此实际查询次数一般小于该上界。

5.7 算法特点

该攻击算法具有以下特点:

- 不需要知道 AES 密钥, 只依赖服务器返回的填充状态;
- 不直接攻击 AES 算法本身, 而是利用 CBC 模式和错误反馈之间的组合漏洞;
- 逐字节恢复中间状态, 再通过异或运算得到明文;
- 攻击过程属于选择密文攻击, 攻击者需要能够向解密服务器提交构造密文并观察反馈。

6 代码实现说明

7 运行结果与破解明文

本节给出程序运行结果和最终破解得到的明文。程序首先对目标密文进行分组, 并检测原始密文是否能够通过服务器的 PKCS#7 填充检查。运行结果显示, 原始密文 padding 正确, 说明解密服务器可以正常访问, Padding Oracle 查询函数可用。

7.1 Oracle 可用性测试

程序运行后首先输出目标密文的分组结果：

```
IV = 46307250616464316e674f7261636c33
C1 = 9ae0735429869542efc40dc3f4c170
C2 = 649463f719c5ddf4ce8c6d1ef0e5d41a
C3 = c5d137629e3fe1340cfaad7e21d65d14
```

随后程序向解密服务器提交原始密文进行测试，服务器返回填充正确：

```
正在测试 oracle...
原始密文 padding 正确，oracle 可用
```

这说明本实验所使用的解密接口能够正常工作，后续可以通过构造密文并观察服务器返回状态进行 Padding Oracle 攻击。

7.2 明文分组恢复结果

程序依次对 C_1 、 C_2 和 C_3 三个密文分组进行攻击。每个分组均从最后一个字节开始向前恢复，逐字节得到对应的中间状态和明文字节。

第 1 个明文分组恢复结果如下：

```
第 1 个明文分组 hex:
666c61677b3535356d73646236617864

第 1 个明文分组 text:
flag{555msdb6axd
```

第 2 个明文分组恢复结果如下：

```
第 2 个明文分组 hex:
3962683065716c76683167666378326b

第 2 个明文分组 text:
9bh0eqlvh1gfcx2k
```

第 3 个明文分组恢复结果如下：

```
第 3 个明文分组 hex:
6379723177733636367d060606060606

第 3 个明文分组 text:
cyr1ws666}
```

其中，第 3 个明文分组末尾的：

06 06 06 06 06 06 (39)

为 PKCS#7 填充字节，表示最后填充了 6 个字节。因此，真实明文不包含这 6 个填充字节。

7.3 完整明文恢复结果

将三个明文分组拼接后，可以得到带有 PKCS#7 填充的完整明文，其十六进制表示为：

666

c61677b3535356d736462366178643962683065716c76683167666378326b6379723177733636

去除末尾 6 个字节的 PKCS#7 填充后，得到真实明文的十六进制表示为：

666

c61677b3535356d736462366178643962683065716c76683167666378326b6379723177733636

将其按 UTF-8 字符串解码，最终得到 5 号目标密文对应的明文为：

flag{555msdb6axd9bh0eq1vh1gfcx2kcyr1ws666}

7.4 运行过程截图说明

为证明攻击过程确实通过逐字节恢复完成，实验报告中应加入程序运行截图。建议至少插入以下两类截图：

- 第一类：Oracle 可用性测试截图，显示目标密文分组以及“原始密文 padding 正确，oracle 可用”；
- 第二类：攻击过程截图，显示程序逐字节输出 fake、intermediate 和 plain 的恢复过程；
- 第三类：最终结果截图，显示带 padding 的明文、去除 padding 后的明文以及最终明文。

8 实验分析与总结